Step-by-Step: Sort data with SQL

This reading outlines the steps the instructor performs in the following video, Sort data with SQL. In the video, the instructor sorts data with SQL using the ORDER BY command. Then, the instructor uses WHERE and ORDER BY together to filter and sort data.

Keep this step-by-step guide open as you watch the video. It can serve as a helpful reference tool if you need additional context or clarification while following the video steps. This is not a graded activity, but you can complete these steps to practice the skills demonstrated in the video.

What you'll need

If you'd like to follow along with the instructor, you will need to log in to your BigQuery account and upload the Movies dataset. To do this, follow the instructions in the reading Upload the movie dataset to BigQuery.

Example 1: Sort data by one column

The `ORDER BY` command sorts data by column in a database. By default, the data is sorted in ascending order.

1. In the BigQuery Explorer pane, select the movie dataset, then the movies table.

2. Select the Preview tab from the Details pane.

3. Select Query then In new tab and enter the following code into the query editor:

```
SELECT *
FROM projectID.movie_data.movies
```

Note: If you're completing this code in BigQuery, replace `projectID` in the code block to your own projectID.

4. Use the `ORDER BY` command to sort the data. Enter `ORDER BY` `` `Release Date` ``; (notice the back-ticks, which are used to capture a column name that contains a space) to sort by the `Release Date` column.

5. Your code should now match this code block:

```
SELECT *
FROM projectID.movie_data.movies
ORDER BY `Release Date`;
```

6. Select RUN. The results return all the movies in the database sorted from oldest to newest.

Example 2: Sort data in descending order

To use the `ORDER BY` command to sort data by descending order, specify `DESC` at the end of the `ORDER BY` command.

1. Enter `DESC` after `ORDER BY` `` `Release Date` `` in the query editor.

2. Your code should now match this code block:

```
SELECT *
FROM projectID.movie_data.movies
ORDER BY `Release Date` DESC;
```

3. Select RUN. The results include the same list of movies, this time sorted from newest to oldest.

Example 3: Filter and sort data in descending order

Use **`WHERE`** and **`ORDER BY`** together to filter, then sort, data.

1. Select the beginning of the **`ORDER BY`** line and press Enter (Windows) or Return (Mac) to add a line between the **`FROM`** and **`ORDER BY`** clauses. The **`ORDER BY`** command is written on the last line to ensure all data is sorted.

2. Select the line you added. Enter **`WHERE Genre = "Comedy"`** to filter for rows in which the Genre column exactly matches **`"Comedy"`**.

3. Your code should now match this code block:

```
SELECT *
FROM projectID.movie_data.movies
WHERE Genre = "Comedy"
ORDER BY `Release Date` DESC;
```

4. Select RUN to run the query. The results include only comedy movies sorted from newest to oldest.

Example 4: Filter on two conditions, then sort data in descending order

Use **`WHERE`**, **`AND`**, and **`ORDER BY`** to filter data on two conditions and then sort it.

1. Select the beginning of the **`ORDER BY`** line and press Enter (Windows) or Return (Mac) to add a line between the **`WHERE`** and **`ORDER BY`** clauses.

2. Select the line you added. Enter **`AND Revenue > 300000000`** to add a condition to your query to return only rows where the Revenue column is greater than 300,000,000.

3. Your code should now match this code block:

```
SELECT *
FROM projectID.movie_data.movies
WHERE Genre = "Comedy"
AND Revenue > 300000000
ORDER BY `Release Date` DESC;
```

4. Select RUN. The results are sorted from newest to oldest and include only comedy movies with revenues greater than $300,000,000.